

Advanced Control Flow and Time Complexity

Matteo Fasiolo

Nested Loop

We have seen that if-else statements can be nested

```
if(condition1){  
    if(condition2){  
        // do something  
    } else {  
        // do something else  
    }  
} else {  
    // do something else  
}
```

Similarly, you can write one loop inside another loop.

For example

```
for (int i = 1; i <= 4; i = i+1){  
  
    for (int j = 1; j <= 4; j = j+1){  
        printf("*");  
    }  
  
    printf("\n");  
}
```

For each outer loop iteration, you print "*" 4 times.

You change line at the end of each outer iteration.

Problem: Write a C program which prints out

```
*  
**  
***  
****
```

Let's look at `nested_for.c`.

We can also nest `while` loops.

Early Loop Exit

How to exit a loop early?

Using the `break` statement we exit a loop immediately.

Find the smallest positive integer a that satisfies the inequality $a^2 + a > 321$:

```
int a = 1;
while(1){
    if(a*a + a > 321){
        printf("%d\n", a);
        break; // exit the while loop immediately.
    }
    a = a + 1;
}
```

See the `break.c` file.

NOTE: it would be odd to use a `for()` loop here.

Exercise: go back to forloop.c

```
for(int i = 1; i <= 10; i = i + 1){  
    printf("%d\n", i);  
}
```

and implement the stopping condition using break

```
for(int i = 1; ; i = i + 1){  
    // TODO: you need to add something here  
    printf("%d\n", i);  
}
```

See forloop2.c for the solution.

Skip an Iteration

How do we skip an iteration of a loop?

The continue statement:

- jumps to the next iteration
- skips the rest of the code block to be iterated

Example: printing all odd numbers from 1 to 10

```
for(int i = 1; i < 10; i = i + 1){  
    if(i % 2 == 0){  
        continue;  
    }  
    printf("%d ", i);  
}  
printf("\n");  
// prints 1 3 5 7 9
```

where `a % b` is the remainder after dividing `a` by `b`.

What will the code below print?

```
int i = 1;
while(i < 10){
    if(i % 2 == 0){
        continue;
    }
    printf("%d \n", i);
    i = i + 1;
}
// ???
```

Take a guess:

- A. 1 3 5 7 9
- B. 2 4 6 8
- C. other

For an answer, run the code in `continue.c` and check.

Time complexity: finding primes

Our task: print every prime number from 2 **up to and including** a positive integer n .

For example, if $n = 10$, the output is

```
2 3 5 7
```

Here, n is the **input size** of the problem.

As n increases, the problem becomes harder.

Can two correct programs produce this same output but perform different amounts of work?

Recall: factors and primes

A prime number:

- is a positive integer greater than 1;
- has exactly two factors: 1 and itself.

We can test whether j is a factor of i using

```
 $i \% j == 0$ 
```

where $j \leq i$.

The expression is true when dividing i by j leaves no remainder.

First algorithm: count every factor

```
for i from 1 to n
    numfactors = 0
    for j from 1 to i
        if i can be divided by j
            numfactors = numfactors + 1

    if numfactors equals 2
        print i
```

This is **pseudo-code**: an understandable description of an algorithm, rather than valid C code.

Exam tip: If you can't provide the right code for a question, at least provide pseudo-code.

First algorithm in C

```
for(int i = 1; i <= n; i++){  
    int numfactor = 0;  
    for(int j = 1; j <= i; j++){  
        if(i % j == 0){  
            numfactor = numfactor + 1;  
        }  
    }  
    if(numfactor == 2){  
        printf("%d ", i);  
    }  
}
```

How much work does this algorithm perform?

What should we count?

We will count how many times the algorithm checks

```
i % j == 0
```

This is the main repeated operation in the algorithm.

We count operations rather than seconds because seconds also depend on the computer, compiler and other running programs.

Trace a small input

For $n = 5$:

Candidate i	Values of j checked	Number of checks
1	1	1
2	1, 2	2
3	1, 2, 3	3
4	1, 2, 3, 4	4
5	1, 2, 3, 4, 5	5

Total: $1 + 2 + 3 + 4 + 5 = 15$ divisibility checks.

Count the work for any n

The inner loop performs

$$T(n) = 1 + 2 + \cdots + n = \text{number of terms} \times \text{average} = n \frac{(n + 1)}{2}$$

checks. So:

- $T(100) = 5050$
- $T(1,000) = 500500$

Increasing n tenfold makes the amount of work approximately one hundred times larger.

From operation counts to Big-O

Time complexity $T(n)$ describes how the amount of work grows as the input size grows.

Here,

$$T(n) = n \frac{n+1}{2} = \frac{n^2 + n}{2}.$$

Big-O notation simplifies this growth pattern.

For large n , the n^2 term controls the growth.

We call this **quadratic time**, written $O(n^2)$.

Given a problem size n , we can divide algorithms into several categories based on their time complexities:

- Constant time: $O(1)$.
- Linear time: $O(n)$.
- Quadratic time: $O(n^2)$.
- Exponential time: $O(2^n)$.

Complexities describe how the work performed by an algorithm grows with n .

Big-O notation omits details that do not control the growth.

For example, if

$$T(n) = 2^n + 7n^2 + 3n,$$

the algorithm is $O(2^n)$.

Back to prime-counting

To improve the algorithm we can stop early.

As soon as we find > 2 factors, we know i is not a prime.

```
if(i % j == 0){  
    numfactor = numfactor + 1;  
    if(numfactor > 2){  
        break;  
    }  
}
```

Note:

- For a composite number such as 12, we can stop early.
- For a prime number such as 13, we do not find a third factor, so we cannot stop early.

For $n = 100$:

- checking every factor performs 5,050 i % j checks;
- using break performs 1,890 checks.

This is a useful speed improvement for this input.

However, a candidate might still require a number of checks proportional to its size.

Fewer checks does not necessarily mean a different Big-O class.

For example, the algorithm might now be $\frac{1}{10} \frac{n(n+1)}{2}$, so still $O(n^2)$.

Can we go below $O(n^2)$?

Consider the factor pairs of 36:

1 x 36 2 x 18 3 x 12 4 x 9 6 x 6

After 6, the same pairs occur in the opposite order:

6 x 6 9 x 4 12 x 3 18 x 2 36 x 1

Since $6 = \sqrt{36}$, we only need to look for a factor up to $\sqrt{36}$.

In general, if i has a factor greater than $\sqrt{i}$, its paired factor must be smaller than $\sqrt{i}$.

Square-root algorithm

```
for(int i = 2; i <= n; i++){  
    int is_prime = 1;  
  
    for(int j = 2; j * j <= i; j++){  
        if(i % j == 0){  
            is_prime = 0;  
            break;  
        }  
    }  
  
    if(is_prime == 1){  
        printf("%d ", i);  
    }  
}
```

Here `is_prime` is a **flag**: 1 means “yes” and 0 means “no”.

Time complexity of the new algorithm

The outer loop examines n candidate numbers.

For each candidate, the inner loop performs at most $\sqrt{n}$ divisibility checks.

The total work is therefore at most

$$n \times \sqrt{n}.$$

The square-root algorithm is $O(n\sqrt{n})$, which grows more slowly than $O(n^2)$.

Compare the algorithms

Number of times $i \% j$ is evaluated:

n	Check every factor	Square root + break
10	55	8
100	5,050	236
1,000	500,500	5,288

Open `prime_complexity.c` in VS Code.

Change `n`, run the program and compare the counts.

Check your understanding

1. What is the input size in this problem?
2. Which operation did we count?
3. If n becomes ten times larger, approximately how much more work does an $O(n^2)$ algorithm perform?
4. Suppose that finding primes up to $n = 10$ takes 1 second, using our first prime-counting algorithm. How long will it take with $n = 100$ (approximately)?
5. Same question, but for the last prime-counting algorithm.

Conclusion

In this lecture, we talked about advanced flow-control:

- Nested loops
- Early loop exit with `break`
- Skipping an iteration with `continue`

We also introduced time-complexity calculations:

- Comparing algorithms by counting repeated operations
- Describing how work grows using Big-O notation